

Segnapunti elettronico per biliardo

Relazione di progetto

Corso: Prototipizzazione con Arduino

Autori: Lisa Giordani e Silvia Bertoni

Docenti: Nicola Dall'Ora e Enrico Fraccaroli

Anno accademico: 2025/2026

Indice

1	Introduzione	2
2	Obiettivi del progetto	2
3	Materiali utilizzati	2
4	Regole di gioco implementate	3
4.1	Modalità 1: segnapunti classico (+1)	3
4.2	Modalità 2: inserimento del valore della bilia	3
5	Architettura del sistema	3
5.1	Ricavo del MAC address	4
6	Funzionamento generale	4
7	Schema del prototipo	4
8	Struttura dati condivisa	5
9	Modulo Sender	6
9.1	Pin e parametri principali	6
9.2	Funzione di lettura della distanza	6
9.3	Invio dell'evento	6
9.4	Inizializzazione dei sensori	7
9.5	Logica di rilevamento nel loop	7
9.6	Codice completo del Sender	7
10	Modulo Receiver	12
10.1	Elementi principali del Receiver	12
10.2	Gestione delle modalità di gioco	12
10.3	Ricezione degli eventi ESP-NOW	13
10.4	Gestione del timer di turno	13
10.5	Assegnazione dei punti e cambio turno	14
10.6	Codice completo del Receiver	14
10.7	Gestione dei falsi positivi	28
10.8	Espandibilità del sistema	29
11	Problematiche riscontrate	30
11.1	Problemi riscontrati con l'ESP8266	30
11.2	Problemi riscontrati con il sensore	30
12	Conclusioni	30
12.1	Estendibilità a sei buche	31

1. Introduzione

Il presente progetto riguarda la realizzazione di un segnapunti elettronico per il biliardo, progettato per rilevare automaticamente l'ingresso delle bilie nelle buche e aggiornare il punteggio del giocatore attivo. Il sistema integra inoltre la gestione del turno, il conteggio del tempo residuo e la visualizzazione in tempo reale delle informazioni principali su display LCD.

Ogni giocatore è identificato da un LED dedicato: il LED acceso indica in modo immediato quale giocatore è attualmente di turno. Il passaggio del turno avviene in modo automatico allo scadere di un timer di 20 secondi, valore scelto per comodità ma facilmente modificabile nel codice qualora si voglia adattare la durata del turno alle esigenze della partita. Durante questi 20 secondi il giocatore di turno ha la possibilità di imbucare una bilia; in caso di imbucata valida il timer viene azzerato e riparte da 20 secondi, consentendo al giocatore di mantenere il turno.

Oltre alla gestione automatica, sono previsti pulsanti per il cambio turno manuale, per l'incremento e il decremento del punteggio e per il reset completo della partita.

2. Obiettivi del progetto

Il sistema è stato progettato con i seguenti obiettivi principali:

- rilevare automaticamente l'imbucata di una bilia tramite sensori a ultrasuoni;
- aggiornare il punteggio del giocatore di turno in modo autonomo;
- indicare chiaramente il giocatore attivo tramite due LED dedicati, uno per ciascun giocatore;
- gestire il cambio turno in modo automatico, allo scadere di un timer di 20 secondi, durante i quali il giocatore ha la possibilità di imbucare una bilia;
- azzerare e far ripartire il timer da 20 secondi ogni volta che viene rilevata un'imbucata valida, così da permettere al giocatore di mantenere il turno finché continua a fare punto;
- gestire il cambio turno in modo manuale tramite apposito pulsante, quando necessario;
- visualizzare su LCD il turno corrente, il tempo residuo del turno e i punteggi dei due giocatori;
- consentire l'incremento e il decremento manuale del punteggio, per correggere eventuali errori di rilevazione o inserimento;
- permettere il reset completo della partita tramite apposito pulsante, riportando il sistema alle condizioni iniziali.

3. Materiali utilizzati

Per la realizzazione del prototipo sono stati utilizzati i seguenti componenti:

- 2 breadboard;
- jumper vari;
- 2 ESP32;
- 1 display LCD 16x2 con interfaccia I2C;
- 2 LED;

- 4 pulsanti;
- 2 sensori HC-SR04;
- 6 resistenze.

4. Regole di gioco implementate

Nel progetto sono state previste due modalità di utilizzo.

4.1 Modalità 1: segnapunti classico (+1)

Nella prima modalità di gioco, definita segnapunti classico, a ogni bilia imbucata viene assegnato automaticamente un punto al giocatore di turno. Il turno rimane attivo finché il giocatore riesce a realizzare almeno una imbucata entro un intervallo di 20 secondi; ogni nuova imbucata valida comporta infatti il riavvio del timer, che riparte nuovamente da 20 secondi. Qualora, invece, il tempo scada senza che venga realizzata alcuna nuova imbucata, il turno passa automaticamente all'altro giocatore. È comunque possibile forzare manualmente il cambio turno tramite l'apposito pulsante.

Per questa modalità il sistema prevede un punteggio massimo pari a 8 punti, soglia scelta in coerenza con la logica di gioco implementata nel codice del Receiver attraverso la costante `MAX_SCORE_M1`. Tale valore rappresenta il limite oltre il quale la partita viene considerata conclusa.

4.2 Modalità 2: inserimento del valore della bilia

Nella seconda modalità, quando viene rilevata un'imbucata, il sistema non assegna immediatamente un punto fisso, ma entra in una schermata di input nella quale l'utente può selezionare, tramite i pulsanti, il valore della bilia imbucata e confermarlo successivamente. Durante questa fase il timer di turno viene temporaneamente congelato, così da non penalizzare il giocatore mentre inserisce il valore corretto. Una volta confermata l'operazione, il punteggio viene aggiornato e il timer riprende il proprio funzionamento.

In questa modalità il punteggio massimo da raggiungere è fissato a 61 punti, valore coerente con la logica del gioco Rotation, nel quale la vittoria in una partita a due giocatori si ottiene comunemente al raggiungimento di 61 punti.

In entrambe le modalità il sistema prevede inoltre la possibilità di decrementare il punteggio in caso di errore e di eseguire il reset completo della partita, riportando il sistema allo stato iniziale.

5. Architettura del sistema

Il sistema è composto da due moduli ESP32 con compiti distinti:

- **ESP32 Sender**, posizionato in prossimità delle buche, che rileva il passaggio delle bilie tramite due sensori a ultrasuoni HC-SR04;
- **ESP32 Receiver**, che riceve gli eventi, aggiorna punteggi e timer, gestisce il turno di gioco, controlla i LED e mostra i dati sul display LCD.

La comunicazione tra i due moduli avviene tramite protocollo ESP-NOW, che consente lo scambio diretto di dati senza l'utilizzo di una rete Wi-Fi tradizionale. Ogni evento inviato dal Sender contiene un identificativo univoco e il numero del sensore che ha rilevato l'imbucata, permettendo così al Receiver di distinguere correttamente i messaggi ricevuti.

5.1 Ricavo del MAC address

Per consentire la comunicazione corretta tramite protocollo ESP-NOW è necessario conoscere il MAC address del dispositivo Receiver, in modo da poterlo inserire nel codice del Sender come indirizzo del nodo destinatario. Nel progetto, il MAC address dell'ESP32 viene ricavato direttamente via software utilizzando la libreria `WiFi.h`, inizializzando il modulo in modalità stazione e stampando poi l'indirizzo sulla porta seriale tramite la funzione `WiFi.STA.macAddress()` oppure `WiFi.macAddress()`.

In questo modo è possibile leggere dal monitor seriale l'indirizzo univoco della scheda e copiarlo successivamente all'interno dell'array `receiverMac[]` usato per la trasmissione dei pacchetti ESP-NOW.

```
1  include <WiFi.h>
2
3  void setup() {
4      Serial.begin(115200);
5      WiFi.mode(WIFI_STA);
6      WiFi.STA.begin();
7      delay(100);
8
9      Serial.print("MAC Address: ");
10     Serial.println(WiFi.STA.macAddress());
11 }
12
13 void loop() {
14 }
```

6. Funzionamento generale

I sensori HC-SR04 misurano continuamente la distanza da eventuali oggetti presenti davanti alle buche. Quando la distanza rilevata scende al di sotto di una soglia prefissata, il sistema interpreta l'evento come il passaggio di una bilia in buca.

In caso di rilevazione valida, il modulo Sender genera un messaggio contenente un identificativo univoco dell'evento e lo invia al modulo Receiver. Quest'ultimo verifica che l'evento non sia duplicato, aggiorna il punteggio del giocatore attivo e riavvia il timer di turno a 20 secondi: in questo modo il giocatore mantiene il turno finché continua a realizzare almeno una imbucata entro la finestra temporale disponibile.

Il Receiver provvede inoltre ad aggiornare il display LCD, mostrando il turno corrente, il tempo residuo e i punteggi dei due giocatori, e a gestire i due LED di stato, mantenendo acceso il LED corrispondente al giocatore di turno. Allo scadere dei 20 secondi senza nuove imbucate, il sistema esegue automaticamente il cambio di turno.

7. Schema del prototipo

In questa sezione possono essere inseriti gli schemi di collegamento del modulo Sender e del modulo Receiver, comprensivi di ESP32, pulsanti, LED, resistenze, display LCD e sensori HC-SR04.

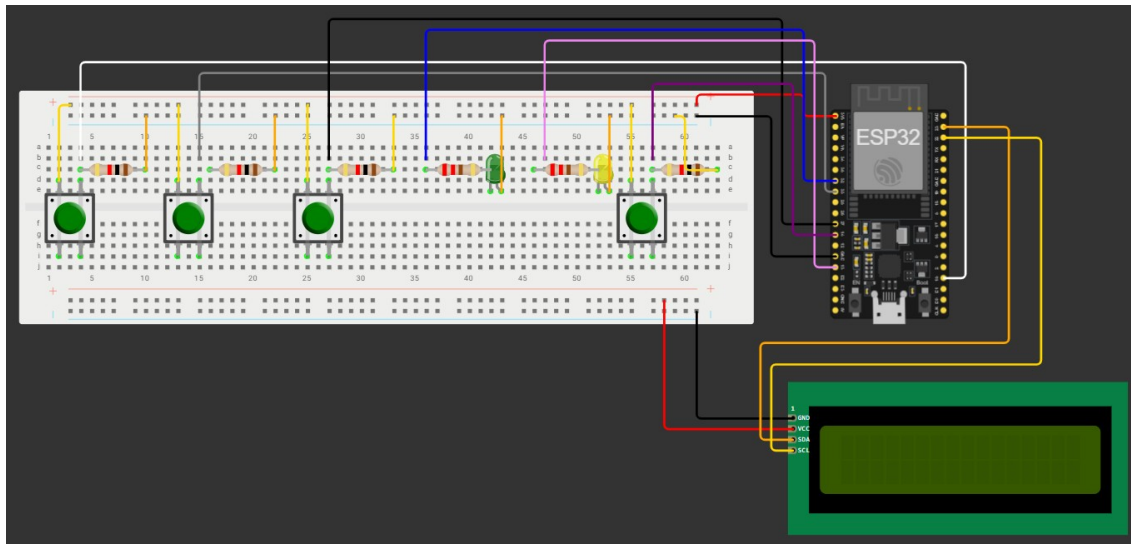


Figura 1: Schema di collegamento del modulo Receiver con ESP32, pulsanti, LED e display LCD.

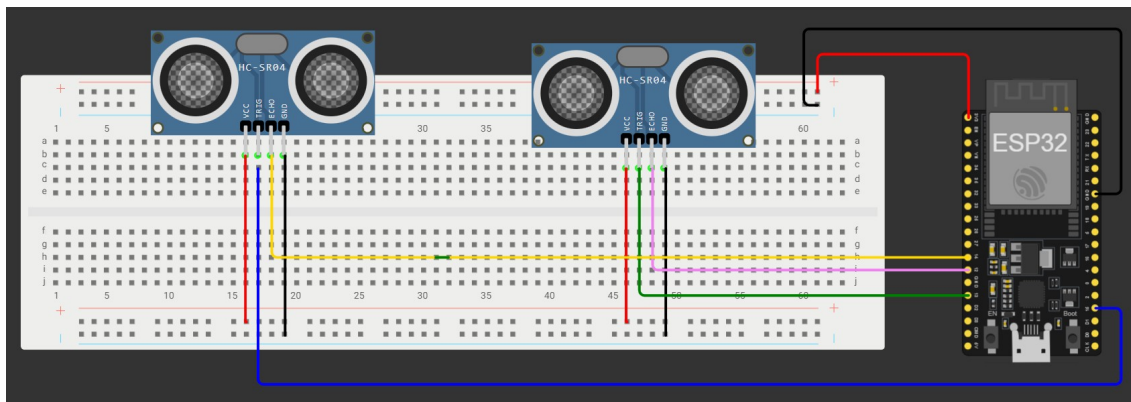


Figura 2: Schema di collegamento del modulo Sender con ESP32 e sensori HC-SR04.

8. Struttura dati condivisa

Per la comunicazione tra i due moduli viene utilizzata una struttura dati comune che contiene un flag di evento, un identificativo progressivo e il numero del sensore che ha rilevato l'imbucata.

Struttura dati per la comunicazione ESP-NOW

```
1 typedef struct struct_message {
2     bool goal;
3     unsigned long id;
4     uint8_t sensorId;
5 } struct_message;
```

Il campo `goal` indica se è stato rilevato un evento valido, il campo `id` rappresenta il numero progressivo dell'evento mentre `sensorId` consente di identificare quale sensore, e quindi quale buca, ha generato il messaggio.

9. Modulo Sender

Il Sender è responsabile della lettura dei sensori, della validazione del rilevamento e della trasmissione del messaggio tramite ESP-NOW.

9.1 Pin e parametri principali

Configurazione del Sender

```
1  const int trig1 = 13;
2  const int echo1 = 12;
3  const int trig2 = 15;
4  const int echo2 = 14;
5  const int ledPin = 2;
6
7  uint8_t receiverMac[] = {0x94, 0x3C, 0xC6, 0x97, 0x4C, 0x78};
8
9  const float triggerDistanceCm1 = 5.0;
10 const float triggerDistanceCm2 = 5.0;
11 const unsigned long sensorLock = 300;
```

9.2 Funzione di lettura della distanza

Letture della distanza tramite sensore HC-SR04

```
1  float readDistanceCm(int trigPin, int echoPin) {
2      digitalWrite(trigPin, LOW);
3      delayMicroseconds(2);
4      digitalWrite(trigPin, HIGH);
5      delayMicroseconds(10);
6      digitalWrite(trigPin, LOW);
7
8      unsigned long durata = pulseIn(echoPin, HIGH, 30000);
9      if (durata == 0) return -1.0;
10     return (durata * 0.0343) / 2.0;
11 }
```

9.3 Invio dell'evento

Invio di un evento di imbucata con identificazione del sensore

```
1  void sendGoalEvent(const char* nomeSensore, uint8_t sid) {
2      dataToSend.goal = true;
3      dataToSend.sensorId = sid;
4      dataToSend.id++;
5
6      esp_err_t result = esp_now_send(receiverMac,
7                                     (uint8_t *)&dataToSend,
8                                     sizeof(dataToSend));
9
10     if (result == ESP_OK) {
11         digitalWrite(ledPin, HIGH);
12     }
```

```

12     delay(80);
13     digitalWrite(ledPin, LOW);
14 } else {
15     Serial.print("Errore esp_now_send(), codice = ");
16     Serial.println(result);
17 }
18 }

```

9.4 Inizializzazione dei sensori

Per evitare falsi positivi all'avvio, nel `setup()` viene eseguita una lettura iniziale dei due sensori. In questo modo, se al momento dell'accensione è già presente un oggetto davanti al sensore, tale condizione non viene interpretata come una nuova imbucata.

Lettura iniziale dei sensori nel setup

```

1 float d1 = readDistanceCm(trig1, echo1);
2 delay(60);
3 float d2 = readDistanceCm(trig2, echo2);
4
5 lastDetected1 = (d1 > 0 && d1 <= triggerDistanceCm1);
6 lastDetected2 = (d2 > 0 && d2 <= triggerDistanceCm2);

```

9.5 Logica di rilevamento nel loop

La rilevazione dell'evento si basa sul fronte di salita del segnale e su un blocco temporale di sicurezza, così da evitare conteggi multipli dovuti al passaggio della stessa bilia.

Rilevamento di una nuova imbucata

```

1 if (detected1 && !lastDetected1 && (now - lastSendTime > sensorLock)) {
2     sendGoalEvent("BUCA 1", 1);
3     lastSendTime = now;
4 }
5
6 if (detected2 && !lastDetected2 && (now - lastSendTime > sensorLock)) {
7     sendGoalEvent("BUCA 2", 2);
8     lastSendTime = now;
9 }

```

9.6 Codice completo del Sender

Sketch completo del modulo Sender

```

1 // =====
2 // SENDER Doppio HC-SR04 + ESP-NOW
3 // Rileva le bilie in due buche e invia:
4 //     goal = true, id univoco, sensorId (1 o 2)
5 // =====
6
7 #include <WiFi.h>      // Libreria Wi-Fi dell'ESP32 (necessaria per ESP-NOW)

```



```

8  #include <esp_now.h>  // Libreria per protocollo ESP-NOW
9
10 // ===== PIN HC-SR04 =====
11 // Coppia TRIG/ECHO per ciascun sensore a ultrasuoni
12 const int trig1 = 13;
13 const int echo1 = 12;
14
15 const int trig2 = 15;
16 const int echo2 = 14;
17
18 // LED di stato sul Sender (lampeggia a ogni invio riuscito)
19 const int ledPin = 2;
20
21 // ===== MAC RECEIVER =====
22 // Indirizzo MAC hardware dell'ESP32 Receiver
23 uint8_t receiverMac[] = {0x94, 0x3C, 0xC6, 0x97, 0x4C, 0x78};
24
25 // ===== SOGLIE =====
26 // Soglia di distanza (in cm) per considerare una bilia presente davanti al sensore
27 const float triggerDistanceCm1 = 5.0;
28 const float triggerDistanceCm2 = 5.0;
29 // Tempo minimo (ms) tra due invii consecutivi per evitare conteggi multipli
30 const unsigned long sensorLock = 300;
31
32 // ===== STRUTTURA DATI =====
33 // Pacchetto che viene inviato al Receiver tramite ESP-NOW
34 // sensorId: 1 = buca 1, 2 = buca 2
35 typedef struct struct_message {
36     bool    goal;      // true se stato rilevato un "goal"
37     unsigned long id; // identificativo univoco incrementale
38     uint8_t sensorId; // identifica quale sensore/buca ha generato l'evento
39 } struct_message;
40
41 // Istanza del pacchetto da inviare
42 struct_message dataToSend;
43 // Struttura di configurazione del peer ESP-NOW
44 esp_now_peer_info_t peerInfo;
45
46 // Stato precedente dei due sensori (per rilevare il fronte di salita)
47 bool lastDetected1 = false;
48 bool lastDetected2 = false;
49 // Timestamp dell'ultimo invio, usato per applicare il sensorLock
50 unsigned long lastSendTime = 0;
51
52 // -----
53 // Misura la distanza in cm con un sensore HC-SR04
54 // Restituisce -1.0 in caso di timeout / assenza di eco
55 float readDistanceCm(int trigPin, int echoPin) {
56     digitalWrite(trigPin, LOW);
57     delayMicroseconds(2);
58     digitalWrite(trigPin, HIGH);
59     delayMicroseconds(10);
60     digitalWrite(trigPin, LOW);
61
62     unsigned long durata = pulseIn(echoPin, HIGH, 30000); // timeout 30 ms
63     if (durata == 0) return -1.0;

```

```

64 // conversione da tempo di volo a distanza (cm)
65 return (durata * 0.0343) / 2.0;
66 }
67
68 // -----
69 // Callback di debug eseguita dopo ogni tentativo di invio ESP-NOW
70 void onDataSent(const wifi_tx_info_t *info, esp_now_send_status_t status) {
71     Serial.print("Stato invio: ");
72     Serial.println(status == ESP_NOW_SEND_SUCCESS ? "OK" : "ERRORE");
73 }
74
75 // -----
76 // Costruisce e invia un pacchetto di "goal" verso il Receiver
77 // - nomeSensore: stringa di debug (es. "BUCA 1")
78 // - sid: id numerico del sensore (1 oppure 2)
79 void sendGoalEvent(const char* nomeSensore, uint8_t sid) {
80     dataToSend.goal = true; // segnala un evento valido
81     dataToSend.sensorId = sid; // memorizza quale sensore ha rilevato
82     dataToSend.id++; // incrementa l'identificativo univoco
83
84     // Messaggio di debug sulla porta seriale
85     Serial.print(nomeSensore);
86     Serial.print(" (buca ");
87     Serial.print(sid);
88     Serial.print(") -> invio ID ");
89     Serial.println(dataToSend.id);
90
91     // Invio del pacchetto al Receiver tramite ESP-NOW
92     esp_err_t result = esp_now_send(receiverMac,
93                                     (uint8_t *)&dataToSend,
94                                     sizeof(dataToSend));
95
96     // Feedback visivo tramite LED in caso di invio riuscito
97     if (result == ESP_OK) {
98         digitalWrite(ledPin, HIGH);
99         delay(80);
100        digitalWrite(ledPin, LOW);
101    } else {
102        Serial.print("Errore esp_now_send(), codice = ");
103        Serial.println(result);
104    }
105 }
106
107 // -----
108 // Inizializzazione del modulo Sender
109 void setup() {
110     Serial.begin(115200);
111     delay(1000);
112
113     Serial.println("=== SENDER DOPPIO HC-SR04 + ESP-NOW ===");
114
115     // Configurazione dei pin dei sensori e del LED
116     pinMode(trig1, OUTPUT);
117     pinMode(echo1, INPUT);
118     pinMode(trig2, OUTPUT);
119     pinMode(echo2, INPUT);

```

```
120     pinMode(ledPin, OUTPUT);
121     digitalWrite(ledPin, LOW);
122
123     // Modalit Wi-Fi in stazione (necessaria per ESP-NOW)
124     WiFi.mode(WIFI_STA);
125     WiFi.STA.begin();
126     delay(100);
127
128     Serial.print("MAC sender: ");
129     Serial.println(WiFi.STA.macAddress());
130
131     // Inizializzazione dello stack ESP-NOW
132     if (esp_now_init() != ESP_OK) {
133         Serial.println("Errore inizializzazione ESP-NOW");
134         return;
135     }
136
137     // Registrazione della callback di invio
138     esp_now_register_send_cb(onDataSent);
139
140     // Configurazione del peer (Receiver) con il suo MAC address
141     memset(&peerInfo, 0, sizeof(peerInfo));
142     memcpy(peerInfo.peer_addr, receiverMac, 6);
143     peerInfo.channel = 0; // canale di default
144     peerInfo.encrypt = false; // nessuna cifratura
145
146     if (esp_now_add_peer(&peerInfo) != ESP_OK) {
147         Serial.println("Errore aggiunta peer");
148         return;
149     }
150
151     // Inizializzazione del pacchetto
152     dataToSend.goal = false;
153     dataToSend.id = 0;
154     dataToSend.sensorId = 0;
155
156     // Lettura iniziale dei sensori per impostare lo stato di partenza
157     float d1 = readDistanceCm(trig1, echol);
158     delay(60); // piccola pausa per evitare interferenza tra i sensori
159     float d2 = readDistanceCm(trig2, echo2);
160
161     // Se una bilia gi davanti al sensore all'avvio,
162     // questa condizione viene salvata ma non genera un evento
163     lastDetected1 = (d1 > 0 && d1 <= triggerDistanceCm1);
164     lastDetected2 = (d2 > 0 && d2 <= triggerDistanceCm2);
165
166     Serial.print("Distanza iniziale sensore 1: ");
167     if (d1 < 0) Serial.println("Fuori portata / timeout");
168     else { Serial.print(d1); Serial.println(" cm"); }
169
170     Serial.print("Distanza iniziale sensore 2: ");
171     if (d2 < 0) Serial.println("Fuori portata / timeout");
172     else { Serial.print(d2); Serial.println(" cm"); }
173
174     Serial.println("Sender pronto.");
175 }
```

```

176
177 // -----
178 // Ciclo principale: lettura sensori, debug e invio eventi
179 void loop() {
180     unsigned long now = millis();
181
182     // ===== SENSORE 1 =====
183     float distanza1 = readDistanceCm(trig1, echo1);
184     // detected1 diventa true se il bersaglio entro la soglia utile
185     bool detected1 = (distanza1 > 0 && distanza1 <= triggerDistanceCm1);
186
187     // Piccolo delay per ridurre interferenze acustiche fra i due HC-SR04
188     delay(60);
189
190     // ===== SENSORE 2 =====
191     float distanza2 = readDistanceCm(trig2, echo2);
192     bool detected2 = (distanza2 > 0 && distanza2 <= triggerDistanceCm2);
193
194     // ===== DEBUG 2 Hz =====
195     // Ogni ~500 ms stampa le distanze e gli stati dei due sensori
196     static unsigned long lastDebug = 0;
197     if (now - lastDebug > 500) {
198         Serial.print("S1 = ");
199         if (distanza1 < 0) Serial.print("timeout");
200         else Serial.print(distanza1);
201         Serial.print(" cm | Det1=");
202         Serial.print(detected1);
203
204         Serial.print(" || S2 = ");
205         if (distanza2 < 0) Serial.print("timeout");
206         else Serial.print(distanza2);
207         Serial.print(" cm | Det2=");
208         Serial.println(detected2);
209
210         lastDebug = now;
211     }
212
213     // Log di debug quando cambia lo stato logico del sensore 1
214     if (detected1 != lastDetected1) {
215         Serial.print("CAMBIO S1: ");
216         Serial.print(lastDetected1);
217         Serial.print(" -> ");
218         Serial.println(detected1);
219     }
220
221     // Log di debug quando cambia lo stato logico del sensore 2
222     if (detected2 != lastDetected2) {
223         Serial.print("CAMBIO S2: ");
224         Serial.print(lastDetected2);
225         Serial.print(" -> ");
226         Serial.println(detected2);
227     }
228
229     // Fronte di ingresso sensore 1:
230     // invia evento SOLO quando si passa da "non rilevato" a "rilevato"
231     // e se passato almeno sensorLock ms dall'ultimo invio

```

```
232     if (detected1 && !lastDetected1 && (now - lastSendTime > sensorLock)) {
233         sendGoalEvent("BUCA 1", 1);
234         lastSendTime = now;
235     }
236
237     // Fronte di ingresso sensore 2
238     if (detected2 && !lastDetected2 && (now - lastSendTime > sensorLock)) {
239         sendGoalEvent("BUCA 2", 2);
240         lastSendTime = now;
241     }
242
243     // Aggiorna lo stato precedente per il ciclo successivo
244     lastDetected1 = detected1;
245     lastDetected2 = detected2;
246
247     // Piccolo delay per limitare la frequenza di polling
248     delay(40);
249 }
```

10. Modulo Receiver

Il Receiver implementa la logica principale del gioco. Gestisce la selezione della modalità all'avvio, il conteggio dei punti, il timer di turno, l'input manuale del valore delle bilie nella seconda modalità, il cambio turno e il reset completo della partita.

10.1 Elementi principali del Receiver

Configurazione iniziale del Receiver

```
1  LiquidCrystal_I2C lcd(0x27, 16, 2);
2
3  const int btnUp    = 33;
4  const int btnDown  = 27;
5  const int btnTurn   = 15;
6  const int btnReset = 14;
7  const int ledP1     = 13;
8  const int ledP2     = 32;
9
10 const int MAX_SCORE_M1 = 8;
11 const int MAX_SCORE_M2 = 61;
12 const unsigned long TURN_TIME = 20000;
```

10.2 Gestione delle modalità di gioco

All'avvio, il sistema mostra sul display una schermata di selezione della modalità. L'utente può evidenziare una delle due opzioni e confermarla tramite pulsante. In questo modo il Receiver viene inizializzato in modo coerente con il tipo di partita che si vuole gestire.

Enumerazioni per modalità e stati di gioco

```

1  enum GameMode { MODE_NONE = 0, MODE_1, MODE_2 };
2  GameMode gameMode = MODE_NONE;
3
4  enum GameState { STATE_SELECT, STATE_PLAY, STATE_INPUT };
5  GameState gameState = STATE_SELECT;

```

10.3 Ricezione degli eventi ESP-NOW

Quando il Receiver riceve un messaggio dal Sender, verifica che la struttura dati sia corretta e che l'identificativo non sia già stato elaborato. Se il messaggio è valido, l'evento viene inserito in una piccola coda circolare, così da poter gestire correttamente eventuali imbucate multiple anche in rapida successione.

Callback di ricezione e coda degli eventi

```

1  void OnDataRecv(const esp_now_recv_info_t *info, const uint8_t *data, int len) {
2      if (len != sizeof(struct_message)) return;
3      memcpy(&incomingData, data, sizeof(incomingData));
4
5      if (incomingData.goal && incomingData.id != lastGoalId) {
6          lastGoalId = incomingData.id;
7          noInterrupts();
8          enqueueGoal(incomingData.sensorId);
9          interrupts();
10     }
11 }

```

10.4 Gestione del timer di turno

Il timer di turno è impostato a 20 secondi. Ogni volta che il giocatore attivo realizza un punto, il timer viene azzerato e riportato a 20 secondi. Nella modalità di inserimento del valore della bilia, invece, il timer viene temporaneamente congelato e poi riattivato una volta completata l'operazione.

Funzioni di gestione del timer

```

1  void pauseTimer() {
2      if (timerPaused) return;
3      turnElapsedBeforePause += millis() - turnStartTime;
4      timerPaused = true;
5  }
6
7  void resumeTimer() {
8      if (!timerPaused) return;
9      turnStartTime = millis();
10     timerPaused = false;
11 }
12
13 void resetTimerTurno() {
14     turnElapsedBeforePause = 0;
15     turnStartTime = millis();
16     timerPaused = false;
17 }

```

10.5 Assegnazione dei punti e cambio turno

Nella prima modalità, ogni evento di imbucata assegna automaticamente un punto al giocatore di turno. Nella seconda modalità, invece, l'evento apre una schermata di input nella quale l'utente seleziona il valore della bilia. Una volta confermato il valore, il punteggio viene aggiornato e il timer riprende.

Funzioni principali di gioco

```

1 void assegnaPunto(int punti) {
2   if (partitaFinita) return;
3
4   if (turnoP1)
5     punteggioP1 = min(punteggioP1 + punti, maxScore());
6   else
7     punteggioP2 = min(punteggioP2 + punti, maxScore());
8
9   resetTimerTurno();
10  aggiornaDisplay();
11 }
12
13 void cambioTurno() {
14   if (!canChangeTurn()) return;
15
16   if (punteggioP1 >= maxScore() || punteggioP2 >= maxScore()) {
17     partitaFinita = true;
18     lastTurnChangeTime = millis();
19     mostraVincitore();
20     return;
21   }
22
23   turnoP1 = !turnoP1;
24   resetTimerTurno();
25   lastTurnChangeTime = millis();
26   aggiornaDisplay();
27 }
```

10.6 Codice completo del Receiver

Sketch completo del modulo Receiver

```

1  // =====
2  // RECEIVER Scoreboard con selezione modalit di gioco
3  //
4  // MODALIT 1 (classica): ogni bilia imbucata = +1 punto
5  // MODALIT 2 (input): quando arriva un goal si entra in
6  //                      modalit inserimento valore; l'utente
7  //                      regola il punteggio con +/- e
8  //                      conferma con btnTurn. Il timer si
9  //                      congela durante l'inserimento.
10 //
11 // SELEZIONE ALL'AVVIO:
12 //   btnUp   -> evidenzia Modalit 1
13 //   btnDown -> evidenzia Modalit 2
14 //   btnTurn -> conferma
15 //
```

```

16 // RESET (in qualsiasi momento) -> torna alla selezione modalit
17 // =====
18
19 #include <Wire.h>           // Comunicazione I2C
20 #include <LiquidCrystal_I2C.h> // Libreria per display LCD I2C
21 #include <WiFi.h>           // Necessaria per usare ESP-NOW su ESP32
22 #include <esp_now.h>        // Libreria protocollo ESP-NOW
23
24 // Oggetto display LCD all'indirizzo I2C 0x27, 16 colonne e 2 righe
25 LiquidCrystal_I2C lcd(0x27, 16, 2);
26
27 // ===== PIN =====
28 // Pulsanti di controllo
29 const int btnUp    = 33;
30 const int btnDown  = 27;
31 const int btnTurn  = 15;
32 const int btnReset = 14;
33
34 // LED che indicano quale giocatore di turno
35 const int ledP1    = 13;
36 const int ledP2    = 32;
37
38 // ===== COSTANTI =====
39 // Modalit 1: punteggio massimo fissato a 8
40 // Modalit 2: punteggio massimo fissato a 61
41 const int MAX_SCORE_M1 = 8;
42 const int MAX_SCORE_M2 = 61;
43
44 // Durata del turno: 20 secondi
45 const unsigned long TURN_TIME = 20000;
46
47 // Tempi di debounce e protezione
48 const unsigned long DEBOUNCE_DELAY = 80; // evita rimbalzi dei pulsanti
49 const unsigned long TURN_CHANGE_LOCK = 700; // evita doppi cambi turno ravvicinati
50 const unsigned long DISPLAY_REFRESH = 300; // frequenza di refresh del display
51
52 // Range ammesso per il valore della bilia nella modalit 2
53 const int MIN_INPUT_VAL = 1;
54 const int MAX_INPUT_VAL = 9;
55
56 // ===== MODALIT DI GIOCO =====
57 // MODE_NONE: nessuna modalit ancora scelta
58 // MODE_1: modalit classica (+1 automatico)
59 // MODE_2: modalit con input del valore della bilia
60 enum GameMode { MODE_NONE = 0, MODE_1, MODE_2 };
61 GameMode gameMode = MODE_NONE;
62 bool modeSelected = false;
63 GameMode highlightedMode = MODE_1;
64
65 // ===== MACCHINA A STATI =====
66 // STATE_SELECT: schermata iniziale di selezione modalit
67 // STATE_PLAY: partita in corso
68 // STATE_INPUT: inserimento manuale del valore della bilia
69 enum GameState { STATE_SELECT, STATE_PLAY, STATE_INPUT };
70 GameState gameState = STATE_SELECT;
71

```



```

72 // ===== STATO GIOCO =====
73 // Punteggi dei due giocatori
74 int punteggioP1 = 0;
75 int punteggioP2 = 0;
76 // true = turno del giocatore 1, false = turno del giocatore 2
77 bool turnoP1 = true;
78 // Indica se la partita terminata
79 bool partitaFinita = false;
80
81 // ===== INSERIMENTO VALORE =====
82 // Valore attualmente selezionato nella schermata di input
83 int inputValore = 1;
84 // true se l'input stato aperto a seguito di un goal ricevuto via ESP-NOW
85 // false se stato aperto manualmente con btnUp
86 bool inputDaGoal = false;
87
88 // ===== BOTTONI =====
89 // Stato precedente dei pulsanti, utile per rilevare il fronte di pressione
90 bool lastUp = HIGH;
91 bool lastDown = HIGH;
92 bool lastTurn = HIGH;
93 bool lastReset = HIGH;
94
95 // Timestamp dell'ultima variazione di stato dei pulsanti
96 unsigned long lastUpTime = 0;
97 unsigned long lastDownTime = 0;
98 unsigned long lastTurnTime = 0;
99 unsigned long lastResetTime = 0;
100
101 // ===== TIMER =====
102 // Istante di partenza del turno corrente
103 unsigned long turnStartTime = 0;
104 // Tempo gi consumato prima di una eventuale pausa
105 unsigned long turnElapsedBeforePause = 0;
106 // true se il timer momentaneamente congelato
107 bool timerPaused = 0;
108 // Timestamp dell'ultimo cambio turno
109 unsigned long lastTurnChangeTime = 0;
110 // Timestamp dell'ultimo aggiornamento del display
111 unsigned long lastDisplayUpdate = 0;
112
113 // ===== ESP-NOW =====
114 // Struttura dei dati ricevuti dal Sender
115 typedef struct struct_message {
116     bool goal; // true se il sender ha rilevato un evento valido
117     unsigned long id; // identificativo progressivo dell'evento
118     uint8_t sensorId; // identifica il sensore che ha generato l'evento
119 } struct_message;
120
121 // Buffer in cui copiare i dati ricevuti
122 struct_message incomingData;
123
124 // Ultimo identificativo elaborato, utile a evitare duplicati
125 volatile unsigned long lastGoalId = 0;
126
127 // ===== CODA CIRCOLARE GOAL =====

```

```

128 // La coda permette di gestire pi eventi ricevuti in rapida successione
129 #define MAX_PENDING 10
130 volatile uint8_t goalQueue[MAX_PENDING];
131 volatile uint8_t goalHead = 0;
132 volatile uint8_t goalTail = 0;
133
134 // =====
135 // HELPER: restituisce la soglia di vittoria in base alla modalit
136 // =====
137 int maxScore() {
138     return (gameMode == MODE_2) ? MAX_SCORE_M2 : MAX_SCORE_M1;
139 }
140
141 // =====
142 // FORWARD DECLARATIONS
143 // Servono a dichiarare in anticipo le funzioni usate nel codice
144 // =====
145 bool pressedEdge(int pinState, bool &lastState,
146                 unsigned long &lastTime, unsigned long now);
147 void resetGioco();
148 void aggiornaDisplay();
149 void mostraSchermataModalita();
150 void mostraSchermataInput();
151 void pauseTimer();
152 void resumeTimer();
153 int tempoResiduoSec();
154
155 // =====
156 // CODA GOAL
157 // Inserisce l'id del sensore nella coda circolare se c' spazio
158 // =====
159 void enqueueGoal(uint8_t sid) {
160     uint8_t next = (goalTail + 1) % MAX_PENDING;
161     if (next != goalHead) {
162         goalQueue[goalTail] = sid;
163         goalTail = next;
164     }
165 }
166
167 // Estrae il prossimo goal in attesa dalla coda
168 bool dequeueGoal(uint8_t &sid) {
169     if (goalHead == goalTail) return false; // coda vuota
170     sid = goalQueue[goalHead];
171     goalHead = (goalHead + 1) % MAX_PENDING;
172     return true;
173 }
174
175 // =====
176 // CALLBACK ESP-NOW
177 // Viene chiamata automaticamente quando arriva un messaggio
178 // =====
179 void OnDataRecv(const esp_now_recv_info_t *info,
180               const uint8_t *data, int len)
181 {
182     // Verifica che la dimensione del pacchetto sia corretta
183     if (len != sizeof(struct_message)) return;

```

```

184
185 // Copia i dati ricevuti nella struttura locale
186 memcpy(&incomingData, data, sizeof(incomingData));
187
188 // Accetta solo i goal validi con id non ancora elaborato
189 if (incomingData.goal && incomingData.id != lastGoalId) {
190     lastGoalId = incomingData.id;
191
192     // Sezione critica: inserimento in coda protetto da interrupt
193     noInterrupts();
194     enqueueGoal(incomingData.sensorId);
195     interrupts();
196 }
197 }
198
199 // =====
200 // TIMER CON PAUSA
201 // Queste funzioni permettono di congelare e riprendere il timer
202 // =====
203
204 // Congela il timer salvando il tempo gi trascorso
205 void pauseTimer() {
206     if (timerPaused) return;
207     turnElapsedBeforePause += millis() - turnStartTime;
208     timerPaused = true;
209 }
210
211 // Riprende il timer dal momento in cui era stato fermato
212 void resumeTimer() {
213     if (!timerPaused) return;
214     turnStartTime = millis();
215     timerPaused = false;
216 }
217
218 // Reinizializza completamente il timer di turno a 20 secondi
219 void resetTimerTurno() {
220     turnElapsedBeforePause = 0;
221     turnStartTime = millis();
222     timerPaused = false;
223 }
224
225 // Calcola i secondi residui del turno
226 int tempoResiduoSec() {
227     unsigned long elapsed = turnElapsedBeforePause;
228     if (!timerPaused) elapsed += millis() - turnStartTime;
229     if (elapsed >= TURN_TIME) return 0;
230     return (TURN_TIME - elapsed + 999) / 1000;
231 }
232
233 // Restituisce true solo se passato abbastanza tempo
234 // dall'ultimo cambio turno
235 bool canChangeTurn() {
236     return (millis() - lastTurnChangeTime) >= TURN_CHANGE_LOCK;
237 }
238
239 // =====

```

```

240 // DISPLAY SELEZIONE MODALIT
241 // Mostra la schermata iniziale in cui scegliere la modalit
242 // =====
243 void mostraSchermataModalita() {
244     lcd.clear();
245     lcd.setCursor(0, 0);
246
247     if (highlightedMode == MODE_1)
248         lcd.print(">MOD 1  MOD 2  ");
249     else
250         lcd.print(" MOD 1 >MOD 2  ");
251
252     lcd.setCursor(0, 1);
253     if (highlightedMode == MODE_1)
254         lcd.print("Classica (+1)  ");
255     else
256         lcd.print("Input valore  ");
257 }
258
259 // =====
260 // DISPLAY INSERIMENTO VALORE
261 // Mostra la schermata usata in modalit 2 per impostare
262 // manualmente il valore della bilia imbucata
263 // =====
264 void mostraSchermataInput() {
265     lcd.clear();
266     lcd.setCursor(0, 0);
267     lcd.print("Valore bilia: ");
268     lcd.setCursor(0, 1);
269     lcd.print("< ");
270     lcd.print(inputValore);
271     lcd.print(" > Conferma ");
272 }
273
274 // =====
275 // DISPLAY PARTITA
276 // =====
277
278 // Mostra il vincitore e il punteggio finale
279 void mostraVincitore() {
280     lcd.clear();
281     lcd.setCursor(0, 0);
282     lcd.print("Vince ");
283     lcd.print(punteggioP1 >= maxScore() ? "P1" : "P2");
284
285     lcd.setCursor(0, 1);
286     lcd.print("P1:");
287     lcd.print(punteggioP1);
288     lcd.print(" P2:");
289     lcd.print(punteggioP2);
290
291     Serial.print("PARTITA FINITA -> Vince ");
292     Serial.println(punteggioP1 >= maxScore() ? "P1" : "P2");
293 }
294
295 // Aggiorna il display principale durante la partita

```

```
296 void aggiornaDisplay() {
297     // Non aggiornare il display di gioco in altri stati
298     if (gameState == STATE_SELECT) return;
299     if (gameState == STATE_INPUT) return;
300
301     // Variabili statiche usate per evitare refresh inutili
302     static int oldP1 = -1;
303     static int oldP2 = -1;
304     static int oldSec = -1;
305     static int oldTurn = -1;
306     static bool oldFine = false;
307
308     int sec = tempoResiduoSec();
309     int turn = turnoP1 ? 1 : 2;
310
311     // Se la partita finita, mostra il vincitore una sola volta
312     if (partitaFinita) {
313         if (!oldFine) {
314             mostraVincitore();
315             oldFine = true;
316         }
317         return;
318     }
319
320     // Se nulla cambiato, evita di ridisegnare il display
321     if (punteggioP1 == oldP1 && punteggioP2 == oldP2 &&
322         sec == oldSec && turn == oldTurn && !oldFine)
323         return;
324
325     // Aggiorna i valori salvati
326     oldFine = false;
327     oldP1 = punteggioP1;
328     oldP2 = punteggioP2;
329     oldSec = sec;
330     oldTurn = turn;
331
332     // Ridisegna il contenuto del display
333     lcd.clear();
334     lcd.setCursor(0, 0);
335     lcd.print("T:");
336     lcd.print(turnoP1 ? "P1" : "P2");
337     lcd.print(" ");
338     // Se il timer in pausa, mostra "***", altrimenti i secondi residui
339     lcd.print(timerPaused ? "***" : String(sec) + "s");
340     lcd.print(" M");
341     lcd.print(gameMode == MODE_1 ? "1" : "2");
342     lcd.print(" ");
343
344     lcd.setCursor(0, 1);
345     lcd.print("P1:");
346     lcd.print(punteggioP1);
347     lcd.print(" P2:");
348     lcd.print(punteggioP2);
349     lcd.print(" ");
350 }
351
```

```
352 // =====
353 // RESET GIOCO
354 // Riporta la partita allo stato iniziale dopo la scelta modalit
355 // =====
356 void resetGioco() {
357     // Svuota la coda degli eventi ricevuti
358     noInterrupts();
359     goalHead = 0;
360     goalTail = 0;
361     lastGoalId = 0;
362     interrupts();
363
364     // Reinizializza le variabili principali della partita
365     punteggioP1 = 0;
366     punteggioP2 = 0;
367     turnoP1 = true;
368     partitaFinita = false;
369     inputValore = 1;
370     gameState = STATE_PLAY;
371
372     // Riavvia timer e lock del cambio turno
373     resetTimerTurno();
374     lastTurnChangeTime = 0;
375
376     Serial.print("RESET GIOCO - Modalita ");
377     Serial.println(gameMode == MODE_1 ? "1 (classica)" : "2 (input valore)");
378
379     lcd.clear();
380     aggiornaDisplay();
381
382     // Allavvio della partita il turno parte dal giocatore 1
383     digitalWrite(ledP1, HIGH);
384     digitalWrite(ledP2, LOW);
385 }
386
387 // =====
388 // ASSEGNA PUNTO
389 // Aggiorna il punteggio del giocatore attivo
390 // =====
391 void assegnaPunto(int punti) {
392     if (partitaFinita) return;
393
394     // Aggiunge i punti al giocatore di turno senza superare il massimo
395     if (turnoP1)
396         punteggioP1 = min(punteggioP1 + punti, maxScore());
397     else
398         punteggioP2 = min(punteggioP2 + punti, maxScore());
399
400     Serial.print("PUNTO +");
401     Serial.print(punti);
402     Serial.print(" -> ");
403     Serial.print(turnoP1 ? "P1" : "P2");
404     Serial.print(" | P1=");
405     Serial.print(punteggioP1);
406     Serial.print(" P2=");
407     Serial.println(punteggioP2);
```

```

408
409 // Ogni punto valido fa ripartire il timer da 20 secondi
410 resetTimerTurno();
411 aggiornaDisplay();
412 }
413
414 // =====
415 // CAMBIO TURNO
416 // Passa il turno all'altro giocatore oppure chiude la partita
417 // =====
418 void cambioTurno() {
419 // Evita doppi cambi ravvicinati
420 if (!canChangeTurn()) return;
421
422 // Se uno dei due giocatori ha raggiunto il punteggio massimo,
423 // la partita termina e viene mostrato il vincitore
424 if (punteggioP1 >= maxScore() || punteggioP2 >= maxScore()) {
425     partitaFinita = true;
426     lastTurnChangeTime = millis();
427     mostraVincitore();
428     return;
429 }
430
431 // Inverte il turno
432 turnoP1 = !turnoP1;
433 // Fa ripartire il timer per il nuovo giocatore
434 resetTimerTurno();
435 lastTurnChangeTime = millis();
436
437 Serial.print("CAMBIO TURNO -> ");
438 Serial.println(turnoP1 ? "P1" : "P2");
439
440 aggiornaDisplay();
441 }
442
443 // =====
444 // ENTRA IN MODALIT INPUT VALORE
445 // Usata nella modalit 2 quando bisogna inserire il valore bilia
446 // =====
447 void entraInInput(bool daGoal) {
448     gameState = STATE_INPUT;
449     inputValore = 1;
450     inputDaGoal = daGoal;
451
452     // Congela il timer mentre l'utente sceglie il valore
453     pauseTimer();
454
455     Serial.println("INIZIO INPUT VALORE");
456     mostraSchermataInput();
457 }
458
459 // =====
460 // CONFERMA INPUT
461 // Termina l'inserimento del valore e torna al gioco
462 // =====
463 void confermaInput() {

```

```

464     gameState = STATE_PLAY;
465
466     // Riattiva il timer dal punto in cui era stato fermato
467     resumeTimer();
468
469     Serial.print("CONFERMATO valore = ");
470     Serial.println(inputValore);
471
472     // Aggiunge al giocatore il valore selezionato
473     assegnaPunto(inputValore);
474
475     // Controlla se il nuovo punteggio porta alla vittoria
476     if (punteggioP1 >= maxScore() || punteggioP2 >= maxScore()) {
477         cambioTurno();
478     }
479
480     aggiornaDisplay();
481 }
482
483 // =====
484 // pressedEdge
485 // Rileva la pressione del pulsante sul fronte HIGH -> LOW
486 // applicando un semplice debounce temporale
487 // =====
488 bool pressedEdge(int pinState, bool &lastState,
489                 unsigned long &lastTime, unsigned long now)
490 {
491     bool pressed = false;
492
493     if (pinState != lastState) {
494         if ((now - lastTime) > DEBOUNCE_DELAY) {
495             lastTime = now;
496
497             // Se il pulsante passa da HIGH a LOW, stato premuto
498             if (pinState == LOW && lastState == HIGH) pressed = true;
499
500             // Aggiorna lo stato precedente
501             lastState = pinState;
502         }
503     }
504     return pressed;
505 }
506
507 // =====
508 // SETUP
509 // Inizializzazione generale del Receiver
510 // =====
511 void setup() {
512     Serial.begin(115200);
513
514     // Configurazione dei pulsanti con resistenza di pull-up interna
515     pinMode(btnUp, INPUT_PULLUP);
516     pinMode(btnDown, INPUT_PULLUP);
517     pinMode(btnTurn, INPUT_PULLUP);
518     pinMode(btnReset, INPUT_PULLUP);
519

```



```

520 // Configurazione LED
521 pinMode(ledP1, OUTPUT);
522 pinMode(ledP2, OUTPUT);
523
524 // Inizializzazione del bus I2C e del display LCD
525 Wire.begin(23, 22);
526 lcd.init();
527 lcd.backlight();
528
529 // Schermata iniziale di avvio
530 lcd.clear();
531 lcd.setCursor(0, 0);
532 lcd.print("Avvio receiver");
533 lcd.setCursor(0, 1);
534 lcd.print("ESP-NOW...");
535 delay(1000);
536
537 // Attiva il modulo Wi-Fi in modalit stazione
538 WiFi.mode(WIFI_STA);
539 WiFi.STA.begin();
540 delay(100);
541
542 Serial.println("=== RECEIVER ESP-NOW ===");
543 Serial.print("MAC receiver: ");
544 Serial.println(WiFi.STA.macAddress());
545
546 // Inizializza ESP-NOW
547 if (esp_now_init() != ESP_OK) {
548     Serial.println("Errore inizializzazione ESP-NOW");
549     lcd.clear();
550     lcd.setCursor(0, 0);
551     lcd.print("Errore ESP-NOW");
552     return;
553 }
554
555 // Registra la callback che gestisce i pacchetti in arrivo
556 esp_now_register_rcv_cb(OnDataRcv);
557 Serial.println("Receiver pronto.");
558
559 // Stato iniziale: selezione modalit
560 gameState = STATE_SELECT;
561 highlightedMode = MODE_1;
562 mostraSchermataModalita();
563 }
564
565 // =====
566 // LOOP
567 // Gestisce input utente, ricezione goal, timer e display
568 // =====
569 void loop() {
570     unsigned long now = millis();
571
572     // Lettura dei pulsanti con debounce e rilevamento del fronte
573     bool upPressed = pressedEdge(digitalRead(btnUp), lastUp, lastUpTime, now);
574     bool downPressed = pressedEdge(digitalRead(btnDown), lastDown, lastDownTime, now);
575     bool turnPressed = pressedEdge(digitalRead(btnTurn), lastTurn, lastTurnTime, now);

```

```
576     bool resetPressed = pressedEdge(digitalRead(btnReset), lastReset, lastResetTime, now);
577
578     // =====
579     // RESET GLOBALE
580     // sempre disponibile e riporta il sistema alla selezione modalit
581     // =====
582     if (resetPressed) {
583         Serial.println("RESET -> selezione modalita");
584
585         noInterrupts();
586         goalHead = 0;
587         goalTail = 0;
588         lastGoalId = 0;
589         interrupts();
590
591         punteggioP1 = 0;
592         punteggioP2 = 0;
593         turnoP1 = true;
594         partitaFinita = false;
595         modeSelected = false;
596         gameMode = MODE_NONE;
597         gameState = STATE_SELECT;
598         highlightedMode = MODE_1;
599
600         resetTimerTurno();
601         mostraSchermataModalita();
602
603         digitalWrite(ledP1, LOW);
604         digitalWrite(ledP2, LOW);
605         return;
606     }
607
608     // =====
609     // STATO: SELEZIONE MODALIT
610     // =====
611     if (gameState == STATE_SELECT) {
612         // btnUp evidenzia la modalit 1
613         if (upPressed) {
614             highlightedMode = MODE_1;
615             mostraSchermataModalita();
616         }
617
618         // btnDown evidenzia la modalit 2
619         if (downPressed) {
620             highlightedMode = MODE_2;
621             mostraSchermataModalita();
622         }
623
624         // btnTurn conferma la modalit selezionata
625         if (turnPressed) {
626             gameMode = highlightedMode;
627             modeSelected = true;
628
629             Serial.print("MODALITA SELEZIONATA: ");
630             Serial.println(gameMode == MODE_1 ? "1 (classica)" : "2 (input valore)");
631         }
632     }
```

```
632     // Mostra brevemente la modalit scelta
633     lcd.clear();
634     lcd.setCursor(0, 0);
635     lcd.print("Modalita ");
636     lcd.print(gameMode == MODE_1 ? "1" : "2");
637     lcd.setCursor(0, 1);
638     lcd.print(gameMode == MODE_1 ? "Classica (+1) " : "Input valore ");
639     delay(1200);
640
641     // Avvia la partita
642     resetGioco();
643 }
644
645 // Durante la selezione modalit i LED restano spenti
646 digitalWrite(ledP1, LOW);
647 digitalWrite(ledP2, LOW);
648 return;
649 }
650
651 // =====
652 // STATO: INSERIMENTO VALORE (solo modalit 2)
653 // =====
654 if (gameState == STATE_INPUT) {
655     bool changed = false;
656
657     // btnUp aumenta il valore selezionato
658     if (upPressed) {
659         if (inputValore < MAX_INPUT_VAL) {
660             inputValore++;
661             changed = true;
662         }
663     }
664
665     // btnDown diminuisce il valore selezionato
666     if (downPressed) {
667         if (inputValore > MIN_INPUT_VAL) {
668             inputValore--;
669             changed = true;
670         }
671     }
672
673     // Se il valore cambiato, aggiorna la schermata
674     if (changed) {
675         Serial.print("Input valore: ");
676         Serial.println(inputValore);
677         mostraSchermataInput();
678     }
679
680     // btnTurn conferma il valore scelto
681     if (turnPressed) {
682         confermaInput();
683         // Eventuali altri goal in coda saranno gestiti nel loop successivo
684     }
685
686     // Anche durante linput i LED continuano a indicare il giocatore di turno
687     if (!partitaFinita) {
```

```
688     digitalWrite(ledP1, turnoP1 ? HIGH : LOW);
689     digitalWrite(ledP2, turnoP1 ? LOW : HIGH);
690 }
691 return;
692 }
693
694 // =====
695 // STATO: GIOCO
696 // =====
697 if (!partitaFinita) {
698
699     // 1. Gestione dei goal ricevuti via ESP-NOW
700     uint8_t sid;
701     noInterrupts();
702     bool hasGoal = dequeueGoal(sid);
703     interrupts();
704
705     if (hasGoal) {
706         if (gameMode == MODE_1) {
707             // Modalit 1: assegna subito +1
708             assegnaPunto(1);
709
710             // Se si raggiunge la soglia, verifica la vittoria
711             if (punteggioP1 >= maxScore() || punteggioP2 >= maxScore())
712                 cambioTurno();
713         } else {
714             // Modalit 2: entra nella schermata di inserimento valore
715             entraInInput(true);
716         }
717     }
718
719     // Se l'arrivo del goal ha spostato il sistema nello stato INPUT, esce
720     if (gameState == STATE_INPUT) {
721         if (!partitaFinita) {
722             digitalWrite(ledP1, turnoP1 ? HIGH : LOW);
723             digitalWrite(ledP2, turnoP1 ? LOW : HIGH);
724         }
725         return;
726     }
727
728     // 2. Pulsante manuale di incremento punteggio
729     if (upPressed) {
730         if (gameMode == MODE_1) {
731             assegnaPunto(1);
732             if (punteggioP1 >= maxScore() || punteggioP2 >= maxScore())
733                 cambioTurno();
734         } else {
735             // In modalit 2 il pulsante apre manualmente linput
736             entraInInput(false);
737         }
738     }
739
740     // 3. Pulsante di decremento punteggio per correggere errori
741     if (downPressed && gameState == STATE_PLAY) {
742         if (gameMode == MODE_1) {
743             if (turnoP1 && punteggioP1 > 0) punteggioP1--;
```

```

744     else if (!turnoP1 && punteggioP2 > 0) punteggioP2--;
745
746     Serial.print("DOWN -1 -> P1=");
747     Serial.print(punteggioP1);
748     Serial.print(" P2=");
749     Serial.println(punteggioP2);
750
751     aggiornaDisplay();
752   } else {
753     // TODO: in modalit 2 si potrebbe implementare una sottrazione
754     // del valore esatto della bilia, con una schermata simile all'input
755   }
756 }
757
758 // 4. Cambio turno manuale
759 if (turnPressed && gameState == STATE_PLAY) {
760   Serial.println("CAMBIO TURNO MANUALE");
761   cambioTurno();
762 }
763
764 // 5. Controllo timeout turno:
765 // se il timer raggiunge i 20 secondi senza nuovi punti, cambia turno
766 if (!timerPaused && (millis() - turnStartTime) + turnElapsedBeforePause >= TURN_TIME) {
767   Serial.print("TIMEOUT -> turno scaduto di ");
768   Serial.println(turnoP1 ? "P1" : "P2");
769   cambioTurno();
770 }
771 }
772
773 // Aggiornamento periodico del display per il conto alla rovescia
774 if ((now - lastDisplayUpdate) >= DISPLAY_REFRESH) {
775   lastDisplayUpdate = now;
776   aggiornaDisplay();
777 }
778
779 // Aggiornamento LED:
780 // se la partita finita, entrambi spenti;
781 // altrimenti acceso solo il LED del giocatore di turno
782 if (partitaFinita) {
783   digitalWrite(ledP1, LOW);
784   digitalWrite(ledP2, LOW);
785 } else {
786   digitalWrite(ledP1, turnoP1 ? HIGH : LOW);
787   digitalWrite(ledP2, turnoP1 ? LOW : HIGH);
788 }
789 }

```

10.7 Gestione dei falsi positivi

Un aspetto importante del progetto riguarda la riduzione dei falsi positivi, cioè rilevazioni errate dovute a rumore del sensore, riflessioni spurie o letture instabili. Per limitare questo problema, nel codice del Sender sono state adottate diverse strategie: l'uso di una soglia minima di rilevazione, il controllo sul fronte di ingresso del segnale, un tempo di blocco tra due invii consecutivi e la gestione del timeout della funzione di misura.

In particolare, l'evento viene trasmesso solo quando il sensore passa dallo stato di non rilevazione allo stato di rilevazione effettiva. In questo modo si evita che la stessa bilia venga conteggiata più volte mentre permane davanti al sensore. Inoltre, il parametro `sensorLock` introduce un intervallo minimo tra due invii consecutivi, riducendo ulteriormente il rischio di conteggi duplicati causati da oscillazioni o misure instabili.

Estratto di codice per la riduzione dei falsi positivi nel Sender

```
1  const float triggerDistanceCm1 = 5.0;
2  const float triggerDistanceCm2 = 5.0;
3  const unsigned long sensorLock = 300;
4
5  float readDistanceCm(int trigPin, int echoPin) {
6      digitalWrite(trigPin, LOW);
7      delayMicroseconds(2);
8      digitalWrite(trigPin, HIGH);
9      delayMicroseconds(10);
10     digitalWrite(trigPin, LOW);
11
12     unsigned long durata = pulseIn(echoPin, HIGH, 30000);
13     if (durata == 0) return -1.0;
14     return (durata * 0.0343) / 2.0;
15 }
16
17 bool detected1 = (distanza1 > 0 && distanza1 <= triggerDistanceCm1);
18
19 if (detected1 && !lastDetected1 && (now - lastSendTime > sensorLock)) {
20     sendGoalEvent("BUCA 1", 1);
21     lastSendTime = now;
22 }
```

10.8 Espandibilità del sistema

Il sistema è stato progettato in modo modulare, così da poter essere esteso con relativa facilità sia dal punto di vista hardware sia dal punto di vista software. L'utilizzo del protocollo ESP-NOW consente infatti una comunicazione wireless diretta tra nodi ESP32, rendendo possibile l'aggiunta di ulteriori moduli Sender o di nuovi sensori senza modificare l'architettura generale del progetto.

Dal punto di vista software, l'espandibilità è favorita dall'uso di una struttura dati dedicata per i messaggi trasmessi, contenente non solo il flag di evento e l'identificativo univoco del pacchetto, ma anche il campo `sensorId`, che permette di distinguere quale sensore abbia generato il dato. Questa scelta rende possibile l'integrazione futura di un numero maggiore di buche, sensori o postazioni di gioco, mantenendo invariata la logica di ricezione lato Receiver.

Estratto di codice che mostra l'espandibilità del sistema

```
1  typedef struct struct_message {
2      bool goal;
3      unsigned long id;
4      uint8_t sensorId;
5  } struct_message;
6
7  void sendGoalEvent(const char* nomeSensore, uint8_t sid) {
8      dataToSend.goal = true;
```

```
9     dataToSend.sensorId = sid;
10    dataToSend.id++;
11
12    esp_now_send(receiverMac,
13                  (uint8_t *)&dataToSend,
14                  sizeof(dataToSend));
15 }
```

11. Problematiche riscontrate

11.1 Problemi riscontrati con l'ESP8266

Nelle fasi iniziali del progetto era stata presa in considerazione una soluzione basata su Arduino ed ESP8266. Tuttavia, sono emersi problemi di comunicazione tra i dispositivi e non è stato possibile ottenere un funzionamento stabile.

Per questo motivo è stata adottata una soluzione interamente basata su ESP32 e protocollo ESP-NOW, che ha semplificato la gestione della comunicazione wireless e ha migliorato l'affidabilità complessiva del sistema. Questa scelta ha inoltre reso più ordinata la suddivisione dei compiti tra modulo di rilevazione e modulo di gestione del punteggio.

11.2 Problemi riscontrati con il sensore

In una prima versione del progetto era previsto l'utilizzo di un sensore a infrarossi (IR) per il rilevamento dell'imbucata. Durante le prove iniziali, con Sender e Receiver montati sulla stessa breadboard e alimentati in modo comune, il sistema sembrava funzionare correttamente: il segnale generato dal sensore IR veniva rilevato e trasmesso senza evidenti anomalie.

Nel passaggio alla configurazione definitiva, con i due moduli fisicamente separati e alimentati in modo indipendente, sono però emerse diverse criticità. In particolare, il sensore IR ha iniziato a mostrare comportamenti instabili e letture poco affidabili, fino al punto in cui il rilevamento non risultava più utilizzabile in maniera ripetibile. Nonostante alcune prove di verifica (controllo dei collegamenti, alimentazione, distanze e soglie di rilevazione), il sistema non ha ripreso a funzionare come nella fase prototipale su breadboard unica.

Considerati i tempi a disposizione e la necessità di garantire un funzionamento robusto, si è deciso di abbandonare l'approccio con sensore IR e di sostituirlo con un sensore a ultrasuoni di tipo HC-SR04. Questa scelta si è rivelata la soluzione più rapida ed efficace per ottenere un rilevamento affidabile dell'imbucata nella configurazione finale del progetto.

12. Conclusioni

Il progetto ha consentito di realizzare un segnapunti elettronico per biliardo capace di automatizzare il rilevamento dell'imbucata e l'aggiornamento del punteggio. La suddivisione tra modulo Sender e modulo Receiver ha reso il sistema ordinato, modulare e facilmente estendibile.

L'integrazione di sensori, display LCD, pulsanti, LED e comunicazione ESP-NOW ha permesso di ottenere un prototipo completo e funzionale, in grado di gestire due differenti modalità di gioco e di adattarsi alle esigenze della partita.

12.1 Estendibilità a sei buche

L'architettura scelta per il sistema consente di estendere il progetto in maniera relativamente semplice fino a coprire tutte le sei buche di un biliardo classico. Dal punto di vista del protocollo di comunicazione, l'uso di ESP-NOW e di una struttura dati dedicata, che include il campo `sensorId`, permette infatti di distinguere in modo univoco l'origine di ciascun evento, indipendentemente dal numero di sensori collegati.

Per passare dalla configurazione attuale (due buche) a una configurazione completa a sei buche sono sufficienti, in linea di principio, i seguenti passi:

- aggiungere sul lato Sender i nuovi sensori (ad esempio altri moduli HC-SR04), definendo i relativi pin `trig` e `echo` e replicando la logica di lettura con soglia per ciascuna nuova buca;
- associare a ogni sensore un `sensorId` distinto (3, 4, 5, 6) e richiamare la funzione di invio `sendGoalEvent(...)` passando l'identificativo corretto in caso di rilevazione valida;
- mantenere invariata la struttura del messaggio ESP-NOW, che già prevede il campo `sensorId`, e il meccanismo di coda sul Receiver, che continua a gestire gli eventi in ingresso in maniera uniforme;
- sul lato Receiver, utilizzare il valore di `sensorId` per eventuali elaborazioni future specifiche per buca (ad esempio statistiche per singola buca o logica differenziata), senza modificare la gestione base del punteggio.

Un esempio semplificato di codice lato Sender per l'aggiunta di un nuovo sensore può essere il seguente:

c

```
1  const int trig3 = 16;
2  const int echo3 = 17;
3
4  void loop() {
5      unsigned long now = millis();
6
7      // Lettura sensore 3
8      float distanza3 = readDistanceCm(trig3, echo3);
9      bool detected3 = (distanza3 > 0 && distanza3 <= triggerDistanceCm1);
10
11     // Fronte di ingresso sensore 3 con lock temporale
12     if (detected3 && !lastDetected3 && (now - lastSendTime > sensorLock)) {
13         sendGoalEvent("BUCA 3", 3);
14         lastSendTime = now;
15     }
16
17     lastDetected3 = detected3;
18 }
```

In questo modo, l'estensione a più sensori avviene principalmente sul modulo Sender, mentre la logica di ricezione e aggiornamento del punteggio lato Receiver rimane sostanzialmente invariata, confermando la modularità e la scalabilità del sistema progettato.